Running the QDK → ALPS DMRG → QPE tests

A step-by-step guide for anyone who has cloned the repository

What these tests show

The code in `examples/interoperability/alps/` takes a molecular Hamiltonian built by QDK/Chemistry, solves it with the ALPS **dmrg** program, and hands the resulting **matrix product state (MPS)** back to QDK. Two test scripts check every step:

Script	What it checks	Run time
check_dmrg_mps.py	DMRG energy and MPS against exact diagonalization and QDK CASCI, on the same (truncated) Hamiltonian.	about 1 minute
check_mps_qpe.py	The MPS loaded into a quantum circuit; quantum phase estimation (QPE) with the **full** Hamiltonian then reproduces QDK CASCI of the full Hamiltonian.	about 22 minutes

The pipeline

```
QDK molecule (SCF + MO integrals)
   |
   |  drop two-body terms over 3 or 4 distinct orbitals   ("truncated H")
   v
ALPS dmrg  (SAVE_MPS=1)  ------------------------------>  MPS tensors
   |
   |  MPS -> determinant expansion -> QDK Wavefunction
   v
QDK state-preparation circuit  (Jordan-Wigner qubits)
   |
   |  quantum phase estimation with the FULL Hamiltonian
   v
energy = QDK CASCI energy of the full Hamiltonian
```

Why the two Hamiltonians differ. For now the handoff drops the two-body integrals (pq|rs) that touch three or four different orbitals, so DMRG solves a *truncated* Hamiltonian. The MPS is therefore not the exact ground state of the real molecule. QPE fixes that: it measures the energy of the *full* Hamiltonian, and the MPS only needs to overlap well with its ground state. That is why the final energy agrees with full CASCI and not with the DMRG energy.

1. What you need

- macOS or Linux, a C++ compiler, CMake 3.20 or newer, Git.
- Boost (1.76 or newer) and HDF5 development files, for building ALPS. On macOS: `brew install cmake boost hdf5`.
- Python 3.10 or newer.
- About 2 GB of disk space and 20-40 minutes for the ALPS build.

2. Get the code

```
git clone https://github.com/marcusr2ML/qdk-chemistry.git
cd qdk-chemistry/examples/interoperability/alps
ls
#  README.md  TESTING_GUIDE.pdf  check_dmrg_mps.py  check_mps_qpe.py  qdk_alps_dmrg.py
```

`qdk_alps_dmrg.py` is the handoff itself; the two `check_*.py` files are the tests.

3. Build ALPS dmrg

The tests need a **dmrg** that can save its MPS (`SAVE_MPS`). That lives on the `dmrg-save-mps-upstream` branch of marcusr2ML/ALPS. Merge in current ALPSim/ALPS master as well: it carries ALPSim/ALPS#221, without which ALPS rounds every coupling to 6 significant digits.

```
cd ~                                    # or wherever you keep builds
git clone --branch dmrg-save-mps-upstream https://github.com/marcusr2ML/ALPS.git alps-mps
cd alps-mps
git pull --no-rebase --no-edit https://github.com/ALPSim/ALPS.git master   # brings in #221

cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
      -DALPS_ENABLE_MPI=OFF -DALPS_BUILD_PYTHON=OFF \
      -DALPS_BUILD_TESTS=OFF -DALPS_BUILD_EXAMPLES=OFF
cmake --build build --target dmrg parameter2xml -j 8
```

Only two programs are built: `dmrg` and `parameter2xml`. Check that the build has SAVE_MPS:

```
ls build/bin/dmrg build/bin/parameter2xml
strings build/bin/dmrg | grep -c SAVE_MPS      # must print a number > 0
```

MPI is switched off on purpose. dmrg does not need it, and Homebrew's Boost ships no `boost_mpi`, which makes the configure step fail with MPI on. If CMake cannot find HDF5, add `-DHDF5_DIR=<prefix>/lib/cmake/hdf5`.

4. Set up Python

```
python3 -m venv ~/qdk-alps-venv
source ~/qdk-alps-venv/bin/activate
pip install qdk-chemistry h5py qiskit
```

`h5py` reads the MPS that dmrg writes; `qiskit` is only used by the QPE test, to read the exact statevector of the circuit. The tests were run with qdk-chemistry 2.1.0.

5. Tell the scripts where ALPS is

Two environment variables, set in the shell you run the tests from:

```
export ALPS_BIN=~/alps-mps/build/bin                    # holds dmrg and parameter2xml
export ALPS_XML_PATH=~/alps-mps/src/alps/resources      # holds ALPS.xsl
```

`ALPS_XML_PATH` is needed because the programs run from the build folder rather than an installed ALPS. If ALPS is installed and on your `PATH`, neither variable is needed.

6. Test 1: DMRG and the MPS

```
cd qdk-chemistry/examples/interoperability/alps
python check_dmrg_mps.py
```

It runs six systems: a Hubbard chain and five small molecules in the STO-3G basis (H_4 at two bond lengths, LiH, an H_6 ring, an H_8 chain). For each, it runs DMRG on the truncated Hamiltonian and compares:

Column	Meaning	Expect
E_dmrg vs E_trunc	DMRG energy vs exact diagonalization of the same truncated Hamiltonian (a plain numpy calculation, independent of ALPS).	|dE| below 1e-12
|d<H>|	The saved MPS, contracted and measured with that matrix, vs E_dmrg. Checks the tensors themselves, not just the printed energy.	below 1e-12
E_casci	QDK's CASCI on the same truncated Hamiltonian.	|dE| below 1e-12
|<CI|MPS>|	Overlap of the MPS, converted to a QDK Wavefunction, with CASCI's.	1.0000000000
E_full, trunc err	CASCI of the full Hamiltonian, and how much the truncation shifts the energy. Informational only.	up to ~0.2 Eh

A passing run ends like this (energies in hartree; columns shortened here):

```
case             dropped        E_dmrg   |dE|       E_casci  |<CI|MPS>|     E_full  trunc err
Hubbard L=6 U=4        0 -3.0925653195 3.2e-14 -3.0925653195 1.0000000000 -3.0925653195  2e-15
H4 chain 0.9 A        72 -2.1980065662 5.3e-15 -2.1980065662 1.0000000000 -2.1803166186 -1.77e-02
H4 chain 1.5 A        72 -1.9324354066 1.3e-14 -1.9324354066 1.0000000000 -1.9961503613  6.37e-02
LiH 1.6 A            312 -7.8956486745 3.6e-15 -7.8956486745 1.0000000000 -7.8823243948 -1.33e-02
H6 ring 1.0 A        408 -3.1972223840 1.1e-14 -3.1972223840 1.0000000000 -3.2374767382  4.03e-02
H8 chain 1.0 A      1776 -4.4811971960 8.9e-15 -4.4811971960 1.0000000000 -4.3075716270 -1.74e-01
DMRG MPS = QDK CASCI ground state of the truncated H
```

7. Test 2: the MPS in a quantum circuit

```
python check_mps_qpe.py          # about 22 minutes
```

This runs H_4 at 0.9 Å and at 1.5 Å (8 qubits each). For each geometry:

- **handoff** - the MPS becomes a QDK Wavefunction and then a state-preparation circuit. The circuit's exact statevector gives $\langle H_{trunc} \rangle$ = E_dmrg, so the state reached the qubits unchanged.
- **JW** - QDK's Jordan-Wigner qubit Hamiltonian (full H) has CASCI's energy as an eigenvalue, with CASCI's wavefunction as the eigenvector (overlap 1).
- **baseline** - $\langle H_{full} \rangle$ measured directly on the MPS state. It sits above CASCI: this is the gap QPE has to close. The line also prints p = $|\langle GS|MPS \rangle|^2$, the chance that a QPE run lands on the ground state.
- **QPE** - three iterative QPE runs (12 phase bits, 15 shots per bit) start from the MPS circuit and use the **full** Hamiltonian. Each must come back within one phase bin of CASCI's full-H energy. The test also confirms that the DMRG energy lies in a different bin, so the agreement cannot be an accident.

Expected result (QPE lines shortened):

```
== H4 chain 0.9 A  (8 qubits, 72 elements dropped for DMRG)
   QPE seed 11: E = -2.178015  (CASCI -2.180317, DMRG -2.198007)  GS
   ...
   ok   baseline <MPS|H_full|MPS> >= E_casci   -2.104223 (+7.61e-02),  p = 0.9425
   ok   QPE      resolution separates E_casci, E_dmrg   bin 5.01e-03 Eh
   ok   QPE      majority of runs on the full-H GS      3/3
```

```
== H4 chain 1.5 A  (8 qubits, 72 elements dropped for DMRG)
   QPE seed 11: E = -1.996682  (CASCI -1.996150, DMRG -1.932435)  GS
   QPE seed 12: E = -1.992726  (CASCI -1.996150, DMRG -1.932435)  GS
   ...
   ok   QPE      majority of runs on the full-H GS      3/3

ALL PASS
```

Reading the QPE numbers. QPE reports energies on a grid; one step ("bin") is about 0.004-0.005 Eh here. A result within one bin of CASCI is a hit. At 1.5 Å CASCI falls between two grid points, so runs can return either neighbour (-1.9967 or -1.9927); both count. The exact values can differ slightly between machines, but the ok/FAIL lines should not.

8. If something goes wrong

Message or symptom	Cause and fix
parameter2xml failed: Cannot find file ALPS.xsl	ALPS_XML_PATH is not set. Point it at <ALPS source>/src/alps/resources (step 5).
dmrg not found in $ALPS_BIN or on PATH	ALPS_BIN is not set or points to the wrong folder.
no /mps in the result; is this dmrg built with SAVE_MPS?	The dmrg on your path is a normal ALPS build. Build the dmrg-save-mps-upstream branch (step 3).
Energies off by about 1e-5 Eh, sometimes below the exact value	ALPS without #221 is rounding the couplings. Do the git pull in step 3 and rebuild.
CMake: Could not find ... boost_mpi	Configure with -DALPS_ENABLE_MPI=OFF.
SAVE_MPS needs at least 4 orbitals	The MPS export needs four or more sites; use a bigger system.
QPE lines say 'not GS'	With a lower overlap p, iterative QPE needs more shots per bit; raise SHOTS_PER_BIT at the top of check_mps_qpe.py.
check_dmrg_mps.py is slow or runs out of memory on a big molecule	The checks build the full Fock-space state (4^L entries). They are meant for small test systems only; the handoff itself has no such limit.

9. Current limits

- Two-body terms over 3 or 4 distinct orbitals are dropped before DMRG. The ALPS `ALPS_n_body_dmrg` branch is meant to lift this.
- Converting the MPS to a QDK Wavefunction goes through the full determinant expansion, so it suits small systems (up to about 10 orbitals).
- Only restricted (closed-shell style) Hamiltonians are accepted.
- dmrg runs with a Lanczos tolerance of 1e-12 by default (ALPS's own default of 1e-7 leaves about 1e-9 Eh on small systems). Change it with `run_dmrg(..., lanczos_tol=...)`.